

Informatik
Hochschule Luzern (Schweiz)

Fazit & Reflexion

Skitourenverwaltung & 3x3-Sicherheitsmatrix

WEBLAB

vorgelegt am Departement Informatik der Hochschule Luzern (Schweiz)

von

Colin Felber

von

Luzern (Schweiz)

Inhaltsverzeichnis

1 Fazit & Reflexion	2
1.1 Retrospektive: Methodischer Ansatz, Zeitbudget und Einarbeitung	2
1.2 Fachlicher und technologischer Abgleich mit der Berufspraxis	2
1.3 Was ist gut gelaufen?	3
1.3.1 Im Code und in der Architektur	3
1.3.2 Im Entwicklungsprozess und beim Arbeiten	3
1.4 Wo lagen die Herausforderungen?	4
1.4.1 KI-Einsatz	4
1.5 Was würde ich das nächste Mal anders resp. besser machen?	5
1.6 Gesamtfazit & Erfüllung der Projektziele	5

1 Fazit & Reflexion

1.1 Retrospektive: Methodischer Ansatz, Zeitbudget und Einarbeitung

Mein Ziel für dieses Modul war es, mir neue Web-Technologien beizubringen. Da ich vorher noch nie mit Angular oder NestJS gearbeitet hatte, war der Einstieg eine grosse Herausforderung und die Lernkurve extrem steil.

Am Anfang haben mir das bereitgestellte Demo-Projekt sowie offizielle Tutorials sehr geholfen. Die eigentliche Hürde war es dann aber, diese Standard-Beispiele auf meine App, die Skitourenverwaltung, zu übertragen. Ich habe in der ersten Woche für einfache Dinge wie Komponenten, Routing oder das State-Management noch enorm viel Zeit gebraucht, wurde ich mit der Zeit immer schneller. Die anfängliche Unsicherheit verschwand und am Ende entwickelte ich eine echte Routine im Umgang mit diesen Web-Technologien.

1.2 Fachlicher und technologischer Abgleich mit der Berufspraxis

Ein besonders prägender und lehrreicher Nebeneffekt der Projektlaufzeit war für mich der zeitliche Umstand, dass ich exakt während dieser Phase den Einstieg in meinen neuen Beruf als Fullstack-Softwareentwickler vollziehen konnte. Dieser parallele Einblick in eine gewachsene Firmen-Codebasis und mein studentisches Projekt ermöglichte mir einen direkten, kritischen Soll-Ist-Vergleich:

- **Komponentengrößen und Modularisierung:** Im beruflichen Alltag sah ich oft monolithische Grosskomponenten, die aus Zeit- und historischen Gründen entstanden waren und die man im Sinne der Wartbarkeit eigentlich herunterbrechen müsste. In meinem WEBLAB-Projekt war es mir daher ein wichtiges Anliegen, von Beginn an durch eine strikte Trennung in Smart-Container und dumb-Components eine saubere, modulare und testbare Struktur aufzubauen.
- **Best Practices aus der Industrie:** Umgekehrt konnte ich wertvolle Best Practices direkt aus dem beruflichen Umfeld in meine Arbeit einfließen lassen, so etwa die konsequente Auslagerung von globalen Design-Variablen und Farben in eine zentrale `colors.css`, um ein konsistentes Look-and-Feel über die gesamte Webapplikation hinweg sicherzustellen und inkonsistentes Styling zu verhindern.

1.3 Was ist gut gelaufen?

Rückblickend bin ich stolz darauf, wie stabil und ausgereift das Gesamtsystem geworden ist, sowohl im Code als auch in meiner methodischen Arbeitsweise:

1.3.1 Im Code und in der Architektur

- **Moderne Komponentenarchitektur & Reactive State (Angular Signals):** Ich habe mich für eine strikte Trennung in zustandslose UI-Komponenten (Dumb Components wie Badge, Card, Button, Map, SecurityMatrixDisplay) und koordinierende Smart-Container entschieden. Durch Angular-Signals (signal, computed, input, effect) und die neue `@if-` / `@for`-Syntax konnte ich einen transparenten Datenfluss etablieren, ohne manuelle Subscription-Verwaltungen zu benötigen.
- **Backend-Modularität & strikte Validierung (NestJS & DTOs):** Im Backend habe ich den Code modular nach Domänen strukturiert (auth, tours, users). Sämtliche Payloads werden über DTOs mit `class-validator` typisiert und strikt validiert, wodurch ungültige Eingaben bereits an der API-Grenze deterministisch mit 400 Bad Request abgefangen werden.
- **Fachliche Domänenlogik (3x3-Sicherheitsmatrix & Leaflet-GPX):** Die digitale 3x3-Sicherheitsmatrix nach Werner Munter konnte ich exakt wie in der alpinen Praxis umsetzen, reaktiv berechnend und fachlich stimmig. Die interaktive Leaflet-Karte liest GPX-Dateien reaktiv ein, visualisiert die Route und berechnet Distanz sowie Höhenmeter dynamisch aus den Rohdaten.
- **Zentrales Design-System:** Mit `colors.css` habe ich ein konsistentes Design-Token-System aufgebaut (Statusfarben für Lawinenstufen, Cards, Typografie), was spätere UI-Anpassungen zentralisiert und Styling-Inkonsistenzen verhindert.
- **Inhaltlich unterschiedlichen Darstellungsformen:** Ich habe in meinen Augen die funktionale Anforderung, dass „die Daten in mindestens zwei inhaltlich unterschiedlichen Darstellungsformen präsentiert werden“ müssen, erfolgreich umgesetzt. Die Touren werden nicht nur in einer klassischen Katalog- bzw. Kachel-Ansicht aufbereitet, sondern zusätzlich interaktiv auf geografischen Karten visualisiert.

1.3.2 Im Entwicklungsprozess und beim Arbeiten

- **Fehlerkultur & methodisches Debugging:** Bei der Leaflet-GPX-Einbindung trat ein Lifecycle-Timing-Problem im DOM auf. Statt unsauberer Workarounds habe ich den Angular-Lifecycle analysiert und die Initialisierung sauber in `ngAfterViewInit` mit anschliessendem `invalidateSize()` gelöst.
- **Steile Lernkurve & beruflicher Wissenstransfer:** Der anfängliche Sprung ins kalte Wasser wandelte sich rasch in ein gutes Verständnis für Angular und NestJS. Welches mir im Beruf geholfen hat, für einen einfacheren Einstieg, wie jedoch auch vom analysieren des Codes im Beruf auch für das Projekt.

1.4 Wo lagen die Herausforderungen?

Im Hinblick auf die Modulanforderungen und die Komplexität der Umsetzung stiess ich an einige signifikante Grenzen:

- **Scope Management & Zeitdruck:** Da ich von Beginn an eine kompromisslose Vision der Applikation im Kopf hatte, geriet ich durch die fixen Abgabefristen zunehmend unter Druck. Da ich mich anfangs in viele neue Framework-Konzepte mühsam einarbeiten musste und manuell nicht so schnell vorankam, wie ich es mir gewünscht hätte, geriet mein ursprünglicher Projektplan ins Wanken.
- **Nachträgliche Security-Integration:** Aus reinem Fokus auf die visuelle Fachlogik hatte ich die Authentifizierung zu Beginn bewusst ausgeklammert und einfach drauflosprogrammiert. Als mir klar wurde, dass sensible Notfallkontakte von Teilnehmenden ein akutes Datenschutzrisiko darstellen, musste ich JWT-Token, Guards und rollenbasierte Zugriffskontrollen nachträglich einziehen. Das rächte sich bitter: Ich musste praktisch jeden Controller, jedes DTO und sämtliche Frontend-Services anfassen. Dieses nachträgliche Hineinflicken von Security kostete mich wertvolle Tage und führte zu grossem Refactoring-Aufwand.
- **Inkonsistente Namenskonventionen:** Im Eifer des Programmierens schlichen sich bei mir anfangs unsaubere Brüche ein, so war beispielsweise teils von `Person` und teils von `User` die Rede, IDs handhabte ich stellenweise als Strings statt als native `MongoDB-ObjectId`, und auch die Ordnerstruktur im Frontend war anfangs nicht konsistent. Diese Altlasten musste ich später mit spürbarem Aufwand bereinigen.

1.4.1 KI-Einsatz

Aufgrund des spürbaren Zeitdrucks traf ich die bewusste Entscheidung, generative KI als Hilfsmittel in meinen Entwicklungsprozess einzubinden. Für mich bestand die zentrale Herausforderung darin, genau abzuwägen, wo ein KI-Einsatz sinnvoll war und wo ich auf meine eigene Programmierleistung setzen musste. Ich nutzte KI vor allem dort, wo ich die zugrundeliegenden Konzepte bereits verstanden hatte und der persönliche Lerneffekt geringer war, beispielsweise bei der Gestaltung und dem visuellen Grundgerüst der Landingpage (Home-Page).

Gleichzeitig war es mir ein grosses Anliegen, generierten Code niemals blind zu übernehmen, sondern jede Zeile eigenhändig zu analysieren und zu verstehen. Ein sehr lehrreiches Beispiel erlebte ich bei der Formatierung des Datums im Schweizer Format (dd/MM/yyyy): Die KI schlug mir hierfür eine unverhältnismässig aufgeblähte, komplizierte Pipeline vor. Durch eigene Recherche und methodische Analyse des Codes erkannte ich die Schwächen dieses Vorschlags und implementierte eigenhändig eine schlanke, performante Lösung, die von der KI nicht vorgeschlagen worden war.

Umgekehrt nutzte ich die KI jedoch auch sehr effektiv als Code-Reviewer. Dies erwies sich als äusserst wertvoll, als ich durch ihr Feedback auf eine kritische Sicherheitslücke aufmerksam gemacht wurde: In der ursprünglichen Logik fehlte eine ausreichende Autorisierungsprüfung, wodurch es theoretisch möglich gewesen wäre, die Daten von jedem beliebigen User unbefugt herunterzuladen. Dank des Reviews konnte ich diese Schwachstelle frühzeitig identifizieren und schliessen.

Zusätzlich nutzte ich die KI als Korrekturhilfe beim Verfassen der arc42 und des Fazits, um meine Rechtschreibung zu verbessern.

Solche Erfahrungen, sowohl das kritische Hinterfragen fehlerhafter KI-Vorschläge als auch die gezielte Nutzung als Fehler-Detektor, schärften meinen analytischen Blick und stellten sicher, dass die architektonische Kontrolle und die volle Code-Hoheit stets bei mir blieben.

1.5 Was würde ich das nächste Mal anders resp. besser machen?

Aus den gemachten Fehlern ziehe ich sehr konkrete und nachhaltige Lehren für meine künftige Arbeit als Softwareentwickler:

- **Security & Auth First:** In künftigen Projekten werde ich Authentifizierung und Berechtigungsstrukturen ausnahmslos an Tag eins aufsetzen. Sicherheit lässt sich in moderne Webarchitekturen schlicht nicht schmerzfrei im Nachhinein hineinflicken.
- **Qualität vor Quantität beim Testing:** Zu Beginn habe ich die Tests bewusst manuell geschrieben, um die Konzepte dahinter wirklich zu verstehen. Besonders spannend war für mich dabei das E2E-Testing (End-to-End), mit dem ich zuvor noch nie gearbeitet hatte. Als ich die Prinzipien von Unit-, Integrations- und E2E-Tests verinnerlicht hatte und der Zeitdruck zunahm, nutzte ich die KI zur Generierung weiterer Tests. Das führte zu einer extremen Menge an Tests, von denen ich viele direkt wieder löschen musste. Ganz zufrieden bin ich mit der Test-Suite aber auch jetzt noch nicht: Viele Frontend-Tests prüfen lediglich, ob statische Elemente korrekt gerendert werden, das bräuchte es in dieser Menge gar nicht. Ich habe mich jedoch bewusst dazu entschieden, diese Tests im Code zu belassen, um transparent zu meinen Fehlern zu stehen und diesen Lernprozess aufzuzeigen.
- **Technologien breiter evaluieren:** Ich habe für dieses Projekt auf das mir vertraute REST gesetzt, bin während der Arbeit aber zum ersten Mal auf GraphQL gestossen und habe mir das genauer angesehen. Für dieses kleine Projekt war REST völlig ausreichend. Bei einem zukünftigen Projekt würde ich mich jedoch nicht mehr automatisch auf REST stürzen, sondern von Beginn an auch solche alternativen Ansätze in Betracht ziehen.

1.6 Gesamtfazit & Erfüllung der Projektziele

Wenn ich auf das WEBLAB-Projekt zurückblicke, bin ich mit dem Ergebnis sehr zufrieden. Die App sieht so aus und funktioniert genau so, wie ich es mir am Anfang vorgestellt habe.

Der Weg dorthin war zwar sehr intensiv. Die Lernkurve für Angular und NestJS war steil, die Zeit knapp und der späte Einbau der Authentifizierung hat mich einiges an Aufwand gekostet. Trotzdem steht jetzt eine moderne und stabile Webapplikation. Das Zusammenspiel aus der Theorie an der Hochschule und den praktischen Eindrücken aus meinem neuen Job als Softwareentwickler hat mir dabei extrem geholfen. Ich konnte sehr viel lernen und nehme aus diesem Modul wertvolle Erfahrungen für meine künftige Arbeit mit. Meine eigenen Ziele sowie die Anforderungen des Moduls habe ich voll und ganz erreicht.